

Keywords: multi-turn conversation • LLM evaluation • performance degradation • conversational AI

LLMs Get Lost In Multi-Turn Conversation

All Frontier Models Suffer a 39% Average Performance Drop Moving from Single-Turn to Multi-Turn Settings

By Philippe Laban, Hiroaki Hayashi, Yingbo Zhou & Jennifer Neville (Salesforce AI Research)

arXiv:2505.06120 • ICLR 2026 Outstanding Paper Award

Every frontier language model tested—open- and closed-weight—performs significantly worse in multi-turn conversations than in equivalent single-turn tasks. Across more than 200,000 simulated conversations spanning six generation tasks, the average performance drop is **39%**. More troublingly, once a model takes a wrong conversational turn, it fails to recover: it gets lost and stays lost.

AT A GLANCE

Problem: LLMs are evaluated almost exclusively in single-turn, fully-specified instruction settings, masking a critical failure mode in multi-turn dialogue.

Why it matters: Real-world use involves extended conversations where user intent is underspecified and refined iteratively; models that degrade are unsafe to deploy in agentic contexts.

Method: 200,000+ simulated conversations on six generation tasks, decomposing the performance gap into aptitude loss and misalignment loss.

Result: 39% average drop across all frontier models; recovery after a wrong turn is essentially zero across all settings.

Evidence: GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro, Llama 3.1 (70B/405B), Mistral Large all exhibit the same pattern.

The Big Picture

LLMs are conversational interfaces by design, yet nearly every benchmark measures performance on single isolated turns with fully specified prompts. This gap matters: analysis of real user logs shows that underspecification is the norm, not the exception. Users iterate, refine, and clarify. The question of how models perform across that iterative process has been largely ignored.

This paper closes that gap with a large-scale simulation study comparing single-turn and multi-turn performance on the same underlying tasks. The results are stark and

consistent: every model tested degrades substantially in the multi-turn setting, and none recovers once a conversational wrong turn has been taken.

Why Existing Approaches Fall Short

Standard benchmarks like MMLU, HumanEval, and MT-Bench evaluate models on isolated, well-formed prompts. Even “multi-turn” benchmarks typically measure whether a model can carry context forward, not whether it can integrate iterative user refinements without losing the original intent.

The underspecification problem is orthogonal to context length: even short conversations (3–5 turns) show significant degradation. This rules out context-length limitations as the primary cause and points instead to a failure in *intent tracking* as conversation state evolves.

Core Mechanism

The authors decompose the multi-turn performance drop into two components:

Aptitude loss: A minor degradation in the model’s core task ability when presented with conversation history rather than a direct prompt—small but measurable (≈ 5 –8 percentage points).

Misalignment loss: The dominant effect (≈ 31 –35 percentage points on average). As the conversation progresses, the model increasingly anchors to its own prior outputs rather than the user’s current intent. A single wrong inference early in the conversation corrupts all downstream turns—the “getting lost” phenomenon.

Non-recovery: The recovery rate R_n —the probability that performance at turn $n + k$ exceeds performance at turn n after a detected misalignment—is approximately zero across all models, tasks, and conversation lengths tested.

Technical Formulation

Let $T = (t_1, t_2, \dots, t_n)$ be a conversation and y the target output. Model performance at turn n is:

$$\text{Perf}_n = P(f(T_{1:n}) = y).$$

The paper observes $\text{Perf}_n < \text{Perf}_1$ for all $n > 1$, all models, all tasks.

The *recovery rate* is defined as:

$$R_n = \frac{1}{K} \sum_{k=1}^K \mathbf{1}[\text{Perf}_{n+k} > \text{Perf}_n].$$

Empirically, $R_n \approx 0$ across all conditions.

The performance gap is decomposed as:

$$\Delta = \underbrace{(\text{Perf}_1 - \text{Perf}_1^{\text{hist}})}_{\text{aptitude loss}} + \underbrace{(\text{Perf}_1^{\text{hist}} - \text{Perf}_n)}_{\text{misalignment loss}}$$

where $\text{Perf}_1^{\text{hist}}$ is single-turn performance when history is present but the core prompt is unchanged.

Learning or Inference Procedure

The evaluation protocol creates multi-turn conversations by simulation:

1. Sample a task instance (x, y) from each benchmark.
2. Construct an underspecified initial prompt t_1 that omits key constraints or details.
3. Simulate a user who provides clarifications at each turn t_k , moving toward the fully-specified target.
4. Score the model’s response at each turn against y .
5. Aggregate across 200,000+ conversations.

What the Guarantee Says

The paper does not provide formal guarantees, but its empirical finding is stated as a universal claim: *no evaluated model* achieves multi-turn performance within 20% of its single-turn performance, and *no evaluated model* has a recovery rate above 5% after a wrong turn.

Experimental Findings

Evaluated on six tasks—summarization, email drafting, code generation, QA, creative writing, data extraction—the paper reports:

- 39% average performance drop across all models and tasks.
- Code generation and QA: largest drops ($\approx 45\%$).
- Creative writing: smallest drop ($\approx 28\%$).
- Closed-weight models (GPT-4o, Claude, Gemini) do not outperform open-weight models (Llama, Mistral) on multi-turn robustness.
- Performance degradation begins at turn 2 and worsens monotonically through turn 5, then plateaus.

Ablations and Interpretation

Turn count: Degradation is sharpest between turns 1 and 2, then increases more slowly. The model’s initial interpretation of the underspecified prompt is highly persistent.

Clarification type: Constraints added at turn 3+ are less reliably incorporated than those present at turn 1. Retroactive clarifications are especially poorly handled.

Model scale: Larger models within the same family show

slightly smaller aptitude loss but equivalent misalignment loss — scale alone does not solve multi-turn robustness.

Implications: The result motivates dedicated multi-turn training objectives, retrieval-augmented intent tracking, and explicit user-intent summarization as architectural additions.

Reference

Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, Jennifer Neville. **LLMs Get Lost In Multi-Turn Conversation**. arXiv:2505.06120, May 2025. <https://arxiv.org/abs/2505.06120>

Keywords: transformer expressivity • succinctness • temporal logic • EXPSPACE-completeness

Transformers are Inherently Succinct

Fixed-Precision Transformers Are Exponentially More Compact Than RNNs and LTL — and Doubly So Versus Finite Automata

By Pascal Bergsträßer, Ryan Cotterell & Anthony W. Lin (ETH Zürich / TU Kaiserslautern)

arXiv:2510.19315 • ICLR 2026 Outstanding Paper Award

Fixed-precision transformers can describe the same languages as linear temporal logic and recurrent neural networks — but exponentially more compactly. Compared with finite automata, the gap is doubly exponential. The price of this compression: verifying even simple transformer properties is EXPSPACE-complete, placing fundamental limits on mechanistic interpretability.

AT A GLANCE

Problem: Prior work shows transformers and classical formalisms can express the same languages, but says nothing about *how compactly*.

Why it matters: Succinctness governs sample efficiency, parameter efficiency, and the computational cost of verification and analysis.

Method: Formal proofs constructing witness language families and providing tight translation bounds between formalism sizes.

Result: Transformers exponentially more succinct than LTL/RNNs; doubly exponentially more succinct than DFAs; verification EXPSPACE-complete.

Evidence: Matching upper and lower bounds — all results are tight.

The Big Picture

Two formalisms that recognize the same languages are not equally useful: one may require a million states to describe what the other captures in ten rules. This notion — *succinctness* — is classical in automata and logic theory, yet was essentially unexplored for transformers.

Prior work on transformer expressivity focused on *what* can be expressed: transformers can simulate circuits, recognize star-free languages, approximate Turing machines. But knowing that an equivalence holds tells you nothing about the cost of translation. This paper fills the gap by providing the first tight succinctness hierarchy for fixed-precision transformers relative to the two most natural

competitors: linear temporal logic (LTL) and recurrent neural networks.

Why Existing Approaches Fall Short

Equivalence proofs (transformer $\equiv$ circuit / LTL / RNN) are the bread and butter of prior expressivity work. They show that the formalisms are inter-translatable, but existing translation procedures blow up doubly exponentially. This was known to be a loose upper bound, but no lower bound was known, leaving open whether a polynomial translation might exist.

This paper closes that gap. The exponential blow-up in translating a transformer to LTL or RNN is not an artifact of proof technique: it is provably unavoidable.

Core Mechanism

The witness construction builds a family of languages $\{L_n\}_{n \geq 1}$:

- **Transformer description:** Size $\text{poly}(n)$ — a constant number of attention heads using positional encodings suffices.
- **LTL formula / RNN:** Any equivalent description requires size $2^{\Omega(n)}$.
- **DFA:** Any equivalent automaton requires size $2^{2^{\Omega(n)}}$.

The key technical insight is that transformers exploit *global self-attention* to encode complex ordering constraints in $O(1)$ heads that LTL must unroll through time and automata must track via exponentially many states. Positional encodings act as a compact coordinate system unavailable to LTL and automata.

Technical Formulation

Main succinctness theorem (informal): There exists a language family $\{L_n\}$ such that:

$$\text{size}(T_n^*) = \text{poly}(n), \quad \text{size}(\phi_n^*) = 2^{\Omega(n)}, \quad \text{size}(A_n^*) = 2^{2^{\Omega(n)}},$$

where T_n^* , ϕ_n^* , A_n^* are the smallest transformer, LTL formula, and DFA for L_n , respectively.

Upper bound (translation theorem): Any fixed-precision transformer with L layers, H heads, and dimension d translates to an LTL formula of size at most $\exp(O(L \cdot H \cdot d))$. This is a single exponential — improving the prior doubly exponential bound.

Verification complexity: The problem

$\text{Model-Check}(M, \varphi) := \text{“Does transformer } M \text{ satisfy property } \varphi\text{?”}$

(for φ an LTL formula) is EXPSPACE-complete. The lower bound follows from the succinctness witness: small transformers can describe exponentially complex behaviours that require EXPSPACE to examine.

Learning or Inference Procedure

This is a purely theoretical paper. The proof proceeds via:

1. **Lower bounds:** Circuit-complexity arguments (communication complexity lower bounds on multi-party protocols) show that any LTL formula or RNN for L_n must have exponential size.
2. **Upper bounds:** Explicit transformer constructions achieving $\text{poly}(n)$ size, and explicit LTL translations achieving single-exponential blow-up from any given transformer.
3. **Complexity:** A reduction from EXPSPACE-complete problems to model-checking of small fixed-precision transformers.

What the Guarantee Says

All bounds are tight:

- The exponential succinctness advantage of transformers over LTL/RNNs is matched by the single-exponential upper bound on translation.
- EXPSPACE-completeness is tight: membership in EXPSPACE follows from the translation theorem; hardness from the reduction.

Experimental Findings

No experiments. All results are formally proved. The paper establishes a complete succinctness hierarchy:

$$\text{Transformer} \prec_{\text{exp}} \text{LTL} \equiv \text{RNN} \prec_{\text{exp}} \text{DFA},$$

where $\prec_{\text{exp}}$ denotes “exponentially more succinct than.”

Ablations and Interpretation

Why succinctness matters for ML: Succinctness directly controls the inductive bias of a hypothesis class. Transformers’ exponential compression advantage explains their remarkable parameter efficiency: they can represent complex structured dependencies in far fewer parameters than classical models.

Implications for interpretability: EXPSPACE-completeness means that mechanistic interpretability — formally verifying that a transformer satisfies a property like “never attends to personal data after a system prompt reset” — is computationally intractable in the worst case. This is not a limitation of current tools; it is a fundamental barrier.

Future directions: The authors conjecture that approximate verification (checking properties of most inputs, not all) may be tractable, and leave open whether practical heuristics can bypass the worst case.

Reference

Pascal Bergsträßer, Ryan Cotterell, Anthony W. Lin. **Transformers are Inherently Succinct.** arXiv:2510.19315, October 2025. <https://arxiv.org/abs/2510.19315>

Keywords: 4D reconstruction • video understanding • feed-forward transformer • dynamic scene analysis

Efficiently Reconstructing Dynamic Scenes One D4RT at a Time

A Single Feedforward Transformer Jointly Infers Depth, Correspondences, and Camera Parameters — 18–300× Faster Than Prior State of the Art

By Chuhan Zhang, Guillaume Le Moing, Skanda Koppula *et al.* (Google DeepMind, UCL, University of Oxford)

arXiv:2512.08924 • CVPR 2026 Best Paper Award

The traditional 4D reconstruction pipeline — depth estimator, optical flow network, SLAM system, fusion module — is replaced by a single feedforward transformer that jointly infers depth, spatio-temporal correspondences, and camera parameters from a monocular video. D4RT achieves 18–300× speedups over specialized baselines while matching or surpassing their accuracy, and was awarded Best Paper at CVPR 2026 from over 16,000 submissions.

AT A GLANCE

Problem: 4D dynamic scene reconstruction requires chaining multiple specialized models, compounding errors and consuming excessive compute.

Why it matters: Autonomous vehicles, AR/VR, and robotic manipulation all need fast, accurate 4D understanding of moving scenes.

Method: A unified transformer with 4D positional encodings and query-based decoding that processes depth, correspondence, and camera estimation in a single forward pass.

Result: 18–300× speedup over per-task specialists; new state-of-the-art on depth, optical flow, and camera estimation.

Evidence: Evaluated on Dynamic Replica, Sintel, DAVIS, Spring, Kubric, and in-the-wild internet videos.

The Big Picture

Recovering the 3D structure and motion of a dynamic scene from video is one of computer vision’s fundamental challenges. The canonical approach chains specialist models: run a monocular depth estimator frame by frame, compute optical flow between adjacent frames with a dedicated flow network, recover camera motion with a SLAM or structure-from-motion system, and then fuse. Each component is trained independently, optimized for its own metric, and blind to the outputs of the others. Errors

accumulate.

D4RT asks: what if a single feedforward network handles all three tasks jointly, letting depth inform correspondence and correspondence constrain camera estimates in a shared representational space? The answer is that it works, and that joint modelling substantially improves each sub-task while eliminating the latency of pipeline execution.

Why Existing Approaches Fall Short

Error compounding: In a pipeline, depth estimation errors propagate into optical flow, which propagates into SLAM. Joint inference breaks this error chain by sharing evidence across tasks.

Computational redundancy: Each specialist extracts its own visual features, producing redundant computation. Sharing a backbone across tasks eliminates this.

Inference latency: Sequential pipeline execution is bounded below by the sum of individual model runtimes. A single feedforward model is bounded only by its own latency, which with modern transformers can be significantly smaller.

Dense decoding cost: Prior feedforward 4D methods decode predictions for all pixels in all frames simultaneously, making inference cost quadratic in video length. D4RT’s query-based decoding is linear in the number of queries requested.

Core Mechanism

Unified token sequence. D4RT embeds all frames into a joint sequence of patch tokens, each annotated with a 4D positional encoding (x, y, t, τ) where x, y are pixel coordinates, t is the frame index, and τ denotes the task modality (depth, correspondence, or camera). A standard transformer stack then lets all tokens attend to each other via full self-attention, sharing information across frames and tasks.

Query-based inference. At test time, the user specifies a set of queries $Q = \{(x_i, y_i, t_i, \tau_i)\}$. Each query cross-attends to the full token sequence, producing a task-specific prediction at that pixel and frame. Inference cost is $O(|Q| \cdot N)$ where N is the total token count, not $O(N^2)$.

Training. All three task losses are applied simultaneously in every forward pass on video data with ground-truth depth, flow, and camera labels.

Technical Formulation

Let $V \in \mathbb{R}^{T \times H \times W \times 3}$ be an input video. Patch embeddings produce context tokens:

$$\mathbf{Z} = \text{PatchEmbed}(V) + \text{PE}_{4D}(x, y, t, \tau),$$

where PE_{4D} is a learned positional encoding across all four dimensions simultaneously.

After transformer processing $\mathbf{Z}' = \text{Transformer}(\mathbf{Z})$, each query $q_i = (x_i, y_i, t_i, \tau_i)$ is decoded via:

$$\hat{y}_i = \text{MLP}(\text{CrossAttn}(q_i, \mathbf{Z}')).$$

The training objective is:

$$\mathcal{L} = \lambda_d \mathcal{L}_{\text{depth}} + \lambda_c \mathcal{L}_{\text{corr}} + \lambda_{\text{cam}} \mathcal{L}_{\text{camera}},$$

where each loss is a task-appropriate combination of L_1 regression and angular/rotation losses, evaluated over the queried subset.

Learning or Inference Procedure

Training proceeds end-to-end on mixed video datasets with ground-truth annotations:

1. Sample a clip of T frames; embed all patches with 4D PE.
2. Sample random query pixels per task modality.
3. Compute joint loss $\mathcal{L}$ across all modalities.
4. Update via AdamW with warmup and cosine decay.

At inference:

1. Embed the input video.
2. Specify queries for desired outputs (dense depth, sparse correspondences, camera poses).
3. Decode predictions in a single forward pass.

What the Guarantee Says

D4RT does not provide formal convergence guarantees. The empirical claim is: a single feedforward model can match or surpass specialized baselines on each individual 4D sub-task while achieving 18–300 $\times$ wall-clock speedup, with the speedup increasing with video length due to query-based decoding’s linear scaling.

Experimental Findings

Evaluated across seven benchmarks:

- **Dynamic Replica:** new state-of-the-art on depth and correspondence metrics.
- **Sintel:** competitive with RAFT on optical flow (EPE within 3%) at 100 $\times$ faster runtime.
- **DAVIS:** strong video object correspondence without domain-specific fine-tuning.
- **Spring & Kubric:** generalizes to synthetic benchmarks outside training distribution.
- **In-the-wild videos:** qualitatively coherent 4D reconstructions of complex internet video.

Speedup range: 18 $\times$ (short clips, dense queries) to 300 $\times$ (long videos, sparse queries).

Ablations and Interpretation

Joint vs. separate training: Joint multi-task training improves each sub-task by 5–12% over single-task training, confirming beneficial task interaction.

4D positional encoding: Removing the frame-index dimension of PE degrades correspondence quality by 18%, confirming that temporal grounding is critical.

Query-based vs. dense decoding: Dense decoding provides marginally higher accuracy (<2%) at 10 $\times$ higher compute. Query-based decoding is preferable for deployment.

Reference

Chuhan Zhang, Guillaume Le Moing, Skanda Koppula, *et al.* **Efficiently Reconstructing Dynamic Scenes One D4RT at a Time.** arXiv:2512.08924, December 2025. <https://arxiv.org/abs/2512.08924>

Keywords: reinforcement learning • maximum likelihood • LLM training • policy gradient

Maximum Likelihood Reinforcement Learning

REINFORCE Optimizes a Lower-Bound Approximation of True Likelihood — MaxRL Fixes This with More Sampling Compute

By Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider, Ruslan Salakhutdinov, Haiwen Feng & Andrea Zanette (Carnegie Mellon University)

arXiv:2602.02710 • ICML 2026 Oral Presentation

Standard reinforcement learning for LLMs does not maximize the likelihood of correct rollouts — it optimizes a Jensen lower bound that allows probability to diffuse across mediocre samples. *Maximum Likelihood Reinforcement Learning* (MaxRL) introduces a compute-indexed family of objectives that converge to exact maximum likelihood as more rollout samples are drawn per query, achieving consistent improvements on math reasoning, navigation, and large-scale Qwen3 training at no increase in gradient steps.

AT A GLANCE

Problem: REINFORCE optimizes only a lower-order approximation of true maximum likelihood over correct rollouts in sampling-based RL settings.

Why it matters: The objective gap causes probability mass to diffuse across mediocre rollouts rather than concentrating on correct answers.

Method: Importance-weighted MLE estimator with k samples per query, interpolating between REINFORCE ($k = 1$) and exact MLE ($k \rightarrow \infty$).

Result: Consistent improvement on maze navigation, GSM8K, and large-scale Qwen3 training; gains scale with k .

Evidence: Toy classification verifies closeness to MLE; ablations confirm the importance-weighting is the key ingredient.

The Big Picture

Training language models with reinforcement learning on binary correctness signals — did the generated code pass the tests? does the math proof check out? — is the dominant paradigm for post-training. The underlying objective is maximum likelihood: given that a correct response exists, maximize the probability the model assigns to it.

But the paper reveals a gap. REINFORCE, the workhorse of this paradigm, does not maximize this likelihood. It optimizes a lower bound, one that is tight only when the model already concentrates most probability on correct responses. In practice, especially early in training, REINFORCE wastes gradient signal on rolling updates that nudge mediocre rollouts upward rather than concentrating on correct ones.

MaxRL closes this gap by drawing $k > 1$ samples per query and constructing an importance-weighted objective whose gradient converges to the true MLE gradient as k grows. More sampling compute buys a closer approximation to the ideal training signal.

Why Existing Approaches Fall Short

REINFORCE: With one sample per query and binary reward, the REINFORCE gradient is:

$$\nabla_{\theta} J_{\text{REINFORCE}} = \mathbb{E}_{y \sim \pi_{\theta}} [r(y) \nabla_{\theta} \log \pi_{\theta}(y|x)].$$

This equals the gradient of $\mathbb{E}_x [\mathbb{E}_{y \sim \pi_{\theta}} [r(y) \log \pi_{\theta}(y|x)]]$, a lower bound on $\mathbb{E}_x [\log \sum_{y \in Y^+} \pi_{\theta}(y|x)]$ by Jensen’s inequality. The gap is non-negligible when π_{θ} spreads probability across many incorrect responses.

Group Relative Policy Optimization (GRPO): Draws k samples but uses them for reward normalization rather than MLE approximation; the resulting objective is still a REINFORCE variant, not a MLE estimator.

Core Mechanism

MaxRL forms an importance-weighted estimate of the true MLE gradient using k rollouts per query. The weights rescale each rollout’s contribution proportionally to the model’s current assignment of probability to that rollout among all correct rollouts sampled:

$$w_i = \frac{r(y_i) \pi_{\theta}(y_i|x)}{\sum_{j=1}^k r(y_j) \pi_{\theta}(y_j|x)}.$$

The MaxRL objective is:

$$J_{\text{MaxRL}}^{(k)}(\theta) = \mathbb{E}_{x, y_{1:k} \sim \pi_{\theta}} \left[\log \frac{1}{k} \sum_{i=1}^k r(y_i) \pi_{\theta}(y_i|x) \right],$$

whose gradient is:

$$\nabla_{\theta} J_{\text{MaxRL}}^{(k)} = \mathbb{E} \left[\sum_{i=1}^k w_i \nabla_{\theta} \log \pi_{\theta}(y_i|x) \right].$$

At $k = 1$ this reduces to REINFORCE; as $k \rightarrow \infty$ this converges to the exact MLE gradient almost surely.

Technical Formulation

Let Y^+ be the set of correct responses for query x . The true maximum likelihood objective is:

$$J_{\text{MLE}}(\theta) = \mathbb{E}_x \left[\log \sum_{y \in Y^+} \pi_\theta(y|x) \right].$$

REINFORCE lower bound: By Jensen’s inequality applied to the concave log,

$$J_{\text{REINFORCE}}(\theta) \leq J_{\text{MLE}}(\theta),$$

with equality only when all probability mass is on a single correct response.

MaxRL convergence:

$$J_{\text{MaxRL}}^{(k)}(\theta) \xrightarrow{k \rightarrow \infty} J_{\text{MLE}}(\theta) \quad \text{a.s.}$$

by the law of large numbers applied to the importance-weighted estimator.

The gap between MaxRL and MLE at finite k is bounded by:

$$|J_{\text{MLE}} - J_{\text{MaxRL}}^{(k)}| = O(k^{-1/2})$$

under mild regularity conditions on π_θ .

Learning or Inference Procedure

1. For each query x , sample k rollouts $y_1, \dots, y_k \sim \pi_\theta(\cdot|x)$.
2. Evaluate reward $r(y_i) \in \{0, 1\}$ for each rollout.
3. Compute importance weights w_i using current policy probabilities.
4. Compute $\nabla_\theta J_{\text{MaxRL}}^{(k)}$ using the weighted gradient estimate.
5. Update θ via Adam.
6. At inference: standard autoregressive sampling (MaxRL changes only the training objective, not the inference procedure).

What the Guarantee Says

MaxRL is a consistent estimator of the MLE gradient: as k grows, the training objective converges to exact maximum likelihood. The rate of convergence is $O(k^{-1/2})$ in the objective value. No distributional shift correction or importance weight clipping is required as long as π_θ remains supported on Y^+ .

Experimental Findings

- **Toy classification:** MaxRL closely tracks exact MLE training signal; REINFORCE lags by a measurable margin throughout.

- **Maze navigation:** MaxRL achieves higher success rate than REINFORCE and GRPO at equivalent gradient steps.
- **GSM8K:** 3–5% accuracy improvement over REINFORCE with the same compute; gains scale with k up to $k = 16$.
- **Qwen3 (large-scale):** Consistent accuracy improvement; MaxRL with $k = 8$ matches REINFORCE with $k = 1$ at roughly half the gradient steps.

Ablations and Interpretation

Effect of k : Performance improves monotonically with k up to $k \approx 16$, beyond which gains plateau (model capacity becomes the bottleneck, not objective quality).

Importance weights vs. uniform: Replacing w_i with uniform weights (equivalent to supervised fine-tuning on correct samples) degrades performance, confirming that the reweighting term is the key ingredient, not merely drawing more samples.

Relationship to GRPO: GRPO’s reward normalization does not approximate MLE; it is a variance-reduction technique within the REINFORCE objective. MaxRL’s improvement is orthogonal and additive to GRPO’s tricks.

Reference

Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider, Ruslan Salakhutdinov, Haiwen Feng, Andrea Zanette. **Maximum Likelihood Reinforcement Learning**. arXiv:2602.02710, February 2026. <https://arxiv.org/abs/2602.02710>

Keywords: robotic manipulation • vision-language-action models • memory benchmarking • long-horizon tasks

RoboMME: Benchmarking and Understanding Memory for Robotic Generalist Policies

16 Tasks Across Four Memory Categories Reveal That Current VLA Memory Mechanisms Are Far From Solving Long-Horizon Manipulation

By Yinpei Dai, Hongze Fu, Jayjun Lee, Yuejiang Liu, Haoran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli & Joyce Chai

arXiv:2603.04639 • ICML 2026 Spotlight Presentation

Vision-language-action models have made remarkable progress on short-horizon manipulation, but long-horizon tasks demanding memory remain poorly understood. RoboMME introduces the first large-scale standardized benchmark for memory in robotic generalist policies, comprising 16 tasks across temporal, spatial, object, and procedural memory categories, and a systematic study of 14 memory-augmented VLA variants. The result: every variant fails significantly on tasks requiring 5+ step sequences, with the oracle gap exceeding 45%.

AT A GLANCE

Problem: VLA evaluations focus on memoryless tasks; no standardized benchmark assesses how well different memory architectures support long-horizon, history-dependent manipulation.

Why it matters: Real-world manipulation involves counting actions, retrieving occluded objects, and executing multi-step procedures — all fundamentally memory-dependent.

Method: 16 tasks classified into four memory types; 14 VLA variants built on the $\pi_{0.5}$ backbone systematically covering representation and integration strategy choices.

Result: FrameSamp+Modul achieves best aggregate performance; all variants fail on 5+ step procedural sequences.

Evidence: Evaluated on LIBERO and LIBERO-Plus with procedurally generated variants ensuring evaluation diversity.

The Big Picture

VLA models extend multimodal LLMs with action outputs, achieving impressive generalization across robot platforms and tasks. Yet every established benchmark

— LIBERO, SimplerEnv, MetaWorld — evaluates performance on tasks solvable from the current observation alone, without reference to history. This memoryless evaluation regime is convenient but incomplete.

Real-world manipulation is deeply history-dependent. A robot sorting items must remember which bin each object was assigned to. A robot following a recipe must track which ingredients have already been added. A robot retrieving objects from a drawer must remember where the object was before it was occluded. Without a principled benchmark for these capabilities, the field has no way to measure progress on long-horizon robustness.

RoboMME fills this gap.

Why Existing Approaches Fall Short

Benchmark limitations: Existing benchmarks either test single tasks independently (no temporal dependency) or use cumulative success rate on fixed-length sequences, conflating task difficulty with memory requirement. RoboMME’s taxonomy separates these confounds.

Architecture comparisons: Prior work comparing memory mechanisms (frame stacking, recurrent states, external memory) uses different tasks, backbones, and training procedures, making conclusions non-transferable. RoboMME fixes the backbone ($\pi_{0.5}$) and training recipe, varying only the memory component.

Core Mechanism

Memory taxonomy (four types):

- **Temporal memory:** Track a sequence of subgoals over time (e.g., place five objects in a specified order; count repeated actions).
- **Spatial memory:** Remember and retrieve the location of objects that become temporarily occluded during the episode.
- **Object memory:** Identify and track specific objects across visual changes (different lighting, partial occlusion, displacement).
- **Procedural memory:** Execute a multi-step procedure from an instruction, where the full procedure is not visible in the current observation.

Memory mechanism variants (14): Each variant combines a *representation* (FrameSamp with $k \in \{1, 4, 8\}$ frames; recurrent hidden state; explicit key-value external memory) with an *integration strategy* (early concatenation; modulation of the VLM trunk; late fusion into the action expert). The full factorial design yields 14 meaningful combinations.

Technical Formulation

Memory score: For task τ and memory category c ,

$$S_\tau = \frac{1}{N_\tau} \sum_{i=1}^{N_\tau} \mathbf{1}[\text{episode } i \text{ succeeds}], \quad M_c = \frac{1}{|T_c|} \sum_{\tau \in T_c} S_\tau.$$

Memory dependence gap: For each task τ ,

$$\Delta_\tau = S_\tau(\text{memory agent}) - S_\tau(\text{Markov agent}),$$

where the Markov agent uses only the current observation (history length 1). Tasks with $\Delta_\tau > 0.1$ are classified as genuinely memory-dependent.

Oracle gap: The gap to an oracle with perfect recall of all history,

$$\Delta_\tau^{\text{oracle}} = S_\tau(\text{oracle}) - \max_{\text{variant}} S_\tau(\text{variant}).$$

For procedural memory tasks, $\Delta_\tau^{\text{oracle}} \approx 0.45$ on average — current mechanisms capture at most 55% of the available memory signal.

FrameSamp+Modul cross-attends k sampled historical frames into the VLM backbone via:

$$\mathbf{h}_t = \text{VLM}(o_t) + \text{CrossAttn}(\text{VLM}(o_t), \{h_{t_j}\}_{j=1}^k),$$

where $\{t_j\}$ are selected historical timesteps.

Learning or Inference Procedure

All 14 variants share the same training recipe:

1. Pre-train on heterogeneous robot demonstration data (OXE-Mix).
2. Fine-tune on RoboMME task demonstrations with per-variant memory module.
3. Evaluate on held-out procedurally generated episodes (100 per task).
4. Report S_τ and Δ_τ for all tasks; M_c per category.

What the Guarantee Says

RoboMME does not provide formal guarantees, but establishes two empirical claims with high statistical confidence ($p < 0.01$ across all reported gaps): (1) 12 of 16 tasks show $\Delta_\tau > 0.1$, confirming genuine memory dependence. (2) No variant achieves $S_\tau > 0.6$ on tasks requiring 5+ step procedural sequences.

Experimental Findings

- **FrameSamp+Modul:** Best aggregate M_c across all four categories; particularly strong on temporal counting and time-sensitive tasks.
- **Recurrent:** Best on procedural memory tasks (leverages hidden state compression), but degrades on temporal counting.

- **ExternalMem:** Marginal gains over FrameSamp on most tasks; naive key-value memory is not yet a silver bullet.
- **Dense history (FrameSamp $k = 8$):** Surprisingly weak; long raw history contexts confuse the VLM backbone without selective attention.
- **All variants:** Fail significantly on 5+ step procedural sequences, with oracle gap $> 45\%$.

Ablations and Interpretation

History length k : Performance peaks at $k = 4$ for most tasks; $k = 8$ hurts due to context confusion. Task-adaptive k selection (based on estimated episode length) is a promising direction.

Integration strategy: Modulation consistently outperforms early concatenation and late fusion, suggesting that injecting historical context into intermediate VLM representations is more effective than prepending raw tokens or appending to the action expert.

What memory helps most: Spatial and object memory benefit most from frame sampling; procedural memory benefits most from recurrence. A hybrid design adapting the memory mechanism to the inferred task type is a natural future direction.

Reference

Yinpei Dai, Hongze Fu, Jayjun Lee, Yuejiang Liu, Hao-ran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli, Joyce Chai. **RoboMME: Benchmarking and Understanding Memory for Robotic Generalist Policies.** arXiv:2603.04639, March 2026. <https://arxiv.org/abs/2603.04639>